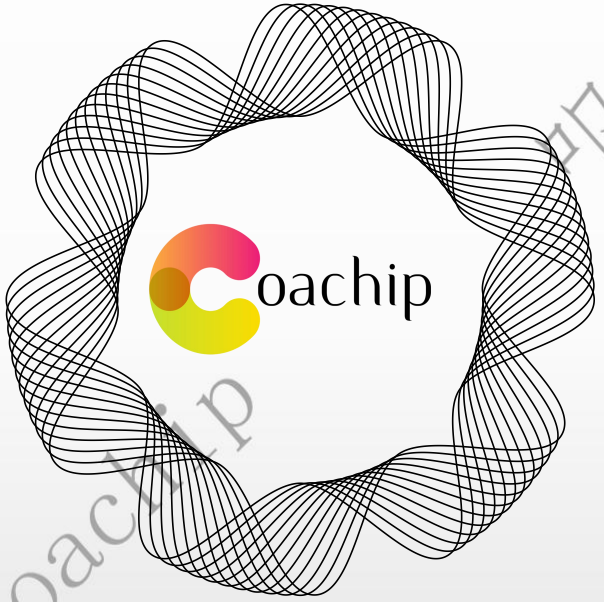


SV的多态

叩特电子Coachip

课程内容及目标

课程内容：

- 了解多态
- 虚方法的概念
- 将子类对象赋值给父类对象（向上类型转换）
- 将指向子类类型的父类指针赋值给子类对象（向下类型转换）

目标

- 了解虚方法的使用
- 了解向上类型转换
- 了解向下类型转换

叩特电子Coachip

多态

当一个类派生出子类的时候，基类中的一些方法可能需要被重写；
用对象中的类型来决定调用**哪一个实现**方法，这是一个动态的过程。

动态的选择方法的实现方式叫多态。
没有继承就没有多态。

叩特电子Coachip

虚方法

(1) 类中的方法在定义的时候通过添加**virtual**关键字来声明一个虚方法,虚方法是一个基本的多态性结构.

(2) 虚方法为具体的实现提供一个原型,也就是在派生类中,重写该方法的时候必须采用**一致的参数和返回值**.

(3) 虚方法可以重写其所有基类中的方法,然而普通的方法被重写后只能在本身及其派生类中有效。

(4) 每个类的继承关系只有一个虚方法的实现,而且是在最后一个派生类中。

叩特电子Coachip

虚方法的例子

```
class packet;
    int a =1;
    int b =2;
    function void display_a;
        $display("packet::a is %d",a);
    endfunction

    virtual function void display_b;
        $display("packet::b is %d",b);
    endfunction
endclass
```

```
class my_packet extends packet;
    int a =3;
    int b =4;
    function void display_a;
        $display("my_packet::a is %d",a);
    endfunction

    virtual function void display_b;
        $display("my_packet::b is %d",b);
    endfunction
endclass
```

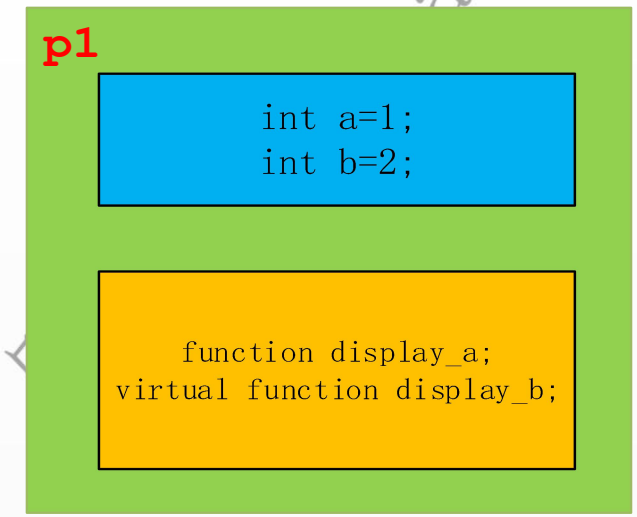
叩特电子Coachip

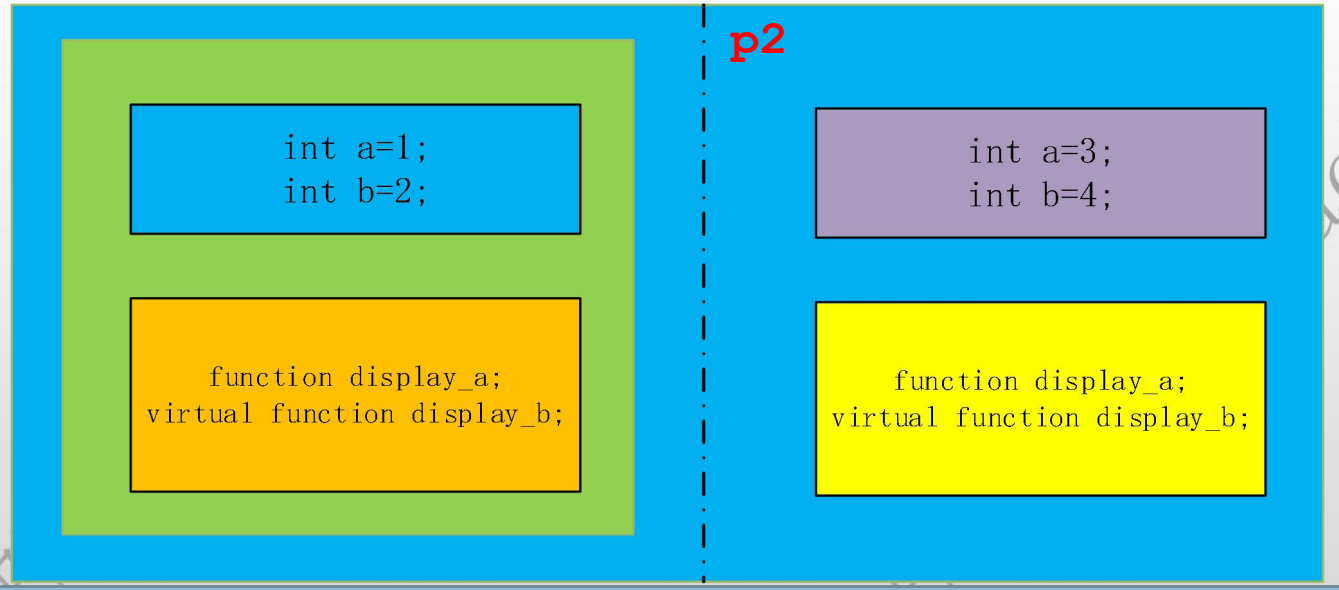
虚方法

```
module top;
    packet p1;
    my_packet p2;

    initial begin
        p1=new();
        p2=new();
        p1 = p2;

        p1.display_a;
        p1.display_b;
    end
endmodule
```





叩特电子Coachip

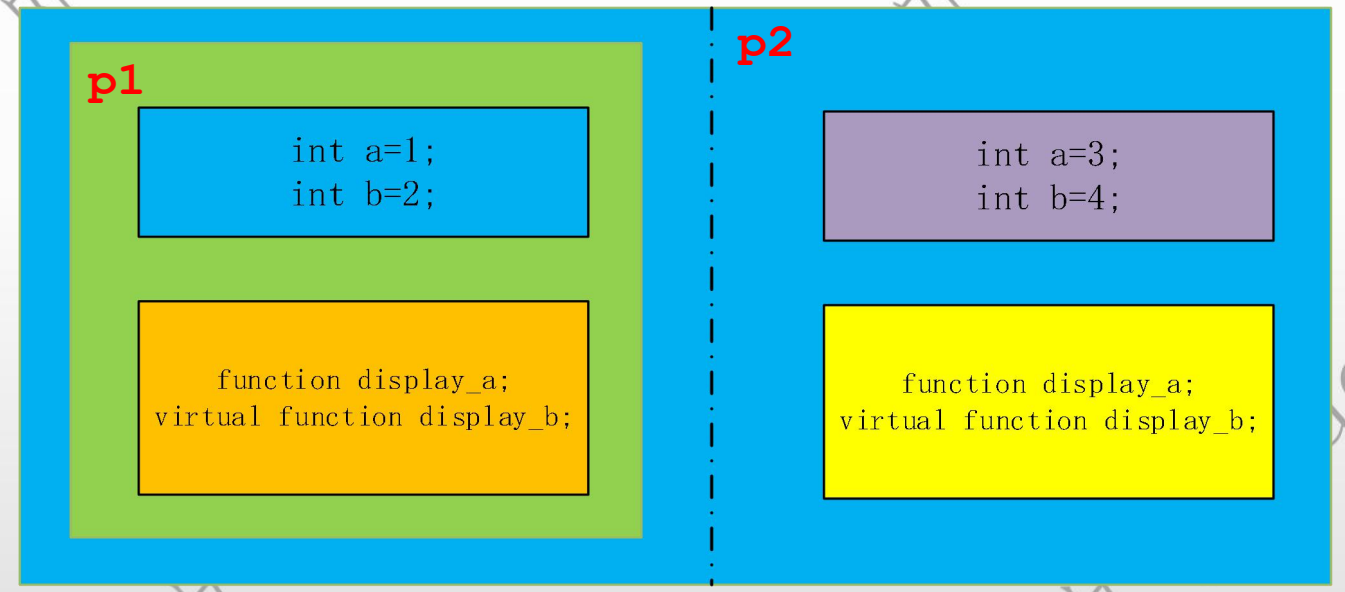
虚方法

```
module top;
    packet p1;
    my_packet p2;

    initial begin
        p1=new();
        p2=new();
        p1 = p2;

        p1.display_a;
        p1.display_b;
    end
endmodule
```

在p1指向p2后，p1只能访问p2所指向空间中的p1的部分。



叩特电子Coachip

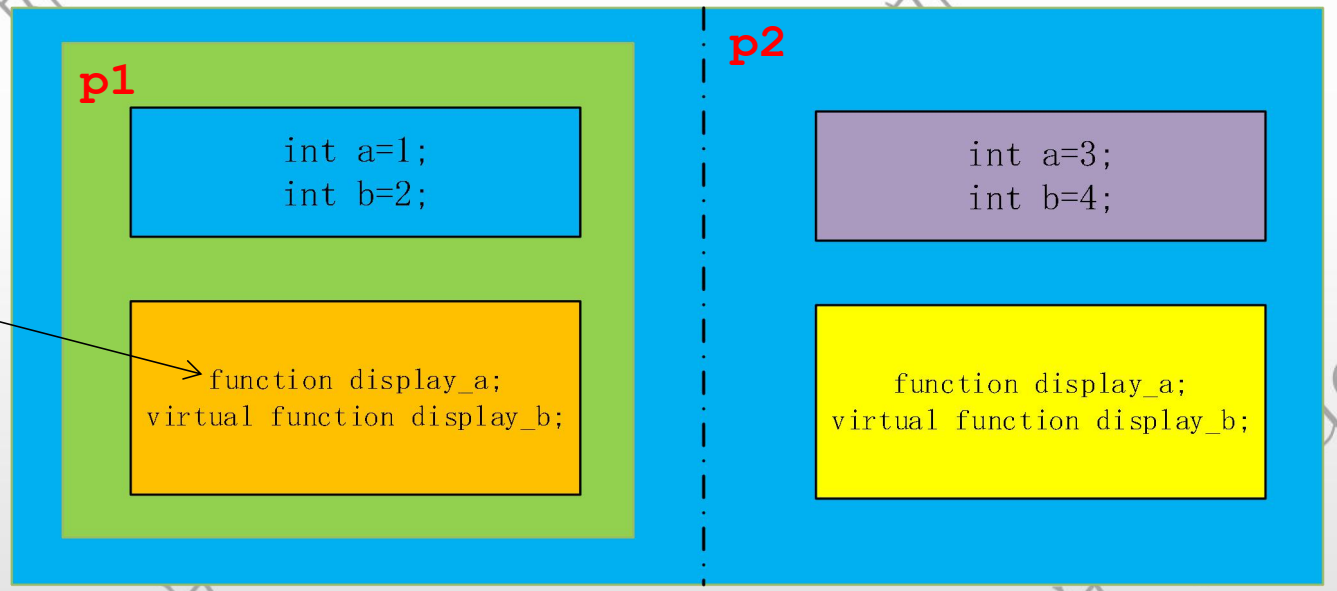
虚方法

```
module top;
    packet p1;
    my_packet p2;

    initial begin
        p1=new();
        p2=new();
        p1 = p2;

        p1.display_a;
        p1.display_b;
    end
endmodule
```

在p1指向p2后,当p1.display_a被调用的时候,程序会调用访问内存中类对象p2中p1的部分,而此时发现display_a是一个普通方法,为此直接调用并结束访问;



即特电子Coachip

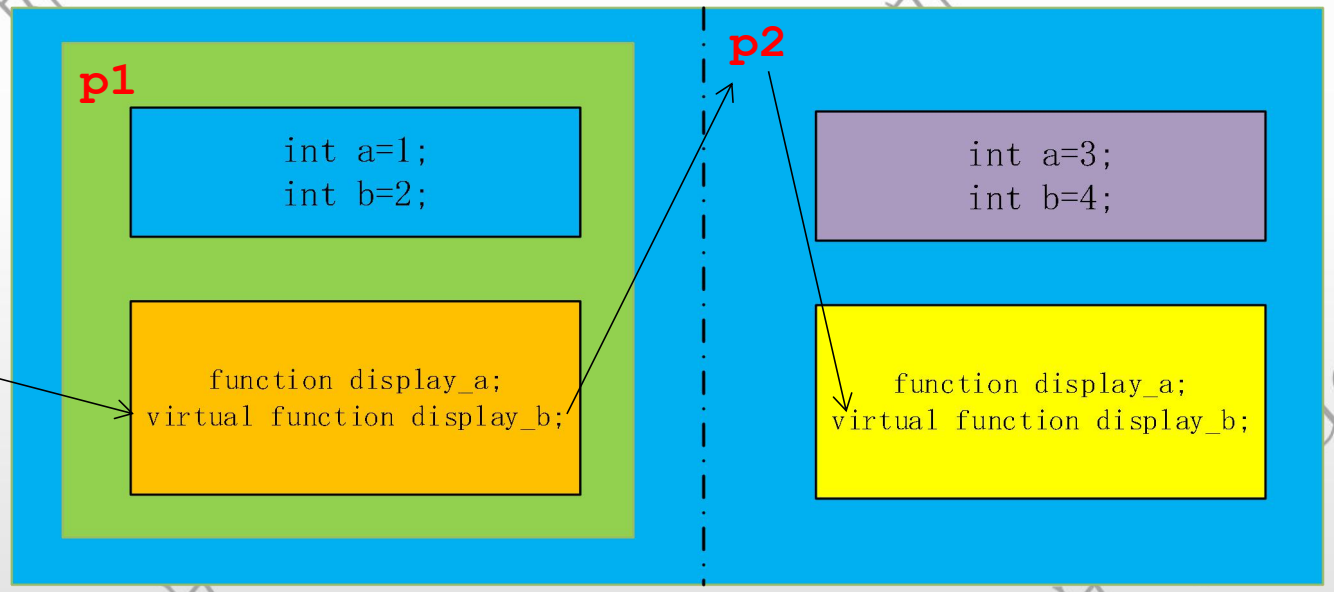
虚方法

```
module top;
    packet p1;
    my_packet p2;

    initial begin
        p1=new();
        p2=new();
        p1 = p2;

        p1.display_a;
        p1.display_b;
    end
endmodule
```

在p1指向p2后,当p1.display_b被调用的时候,程序同样找到类对象p2中p1的部分,此时发现display_b是个虚方法,这时其会咨询系统,查看整个p2在定义的时候是否重写了该方法.系统发现在p2中(除了p1外),确实重写了该方法,为此,程序会直接调用p2重写的实现.



带有虚方法的类称为多态类

叩特电子Coachip

类型转换

类型转换也算一种多态形式：

子类向父类的转换称为向上类型转换；

从父类向子类的转换，称为向下类型转换；

向上类型转换是安全的，而反之则是不安全的。

叩特电子Coachip

向上类型转换

```
class father;
  string m_name;

  function new (string name);
    m_name = name;
  endfunction

  function void print ();
    $display("Hello %s", m_name);
  endfunction
endclass : father
```

```
class child0 extends father;
  string car = "car";

  function new (string name);
    super.new(name);
  endfunction
endclass : child0
```

```
class child1 extends father;
  string plane = "plane";

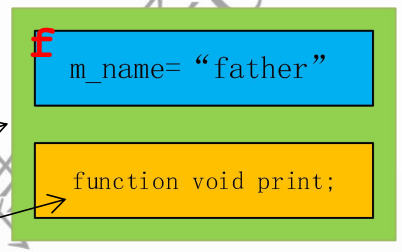
  function new (string name);
    super.new(name);
  endfunction
endclass : child1
```

叩特电子Coachip

向上类型转换

```
module top;
  father f;
  child0 c0;
  child1 c1;

  initial begin
    f = new("father");
    f.print();
    c0 = new("child0");
    f = c0;
    f.print();
    c1 = new("child1");
    f = c1;
    f.print();
  end
endmodule : top
```



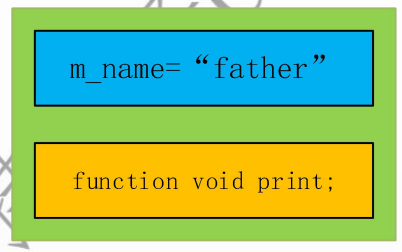

```
Hello father
```

叩特电子Coachip

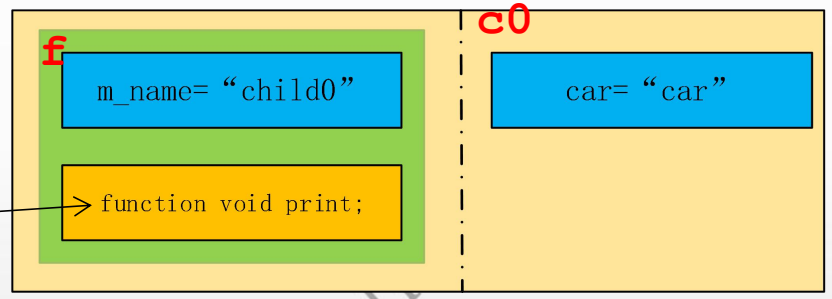
向上类型转换

```
module top;
  father f;
  child0 c0;
  child1 c1;

  initial begin
    f = new("father");
    f.print();
    c0 = new("child0");
    f = c0;
    f.print();
    c1 = new("child1");
    f = c1;
    f.print();
  end
endmodule : top
```



Hello father



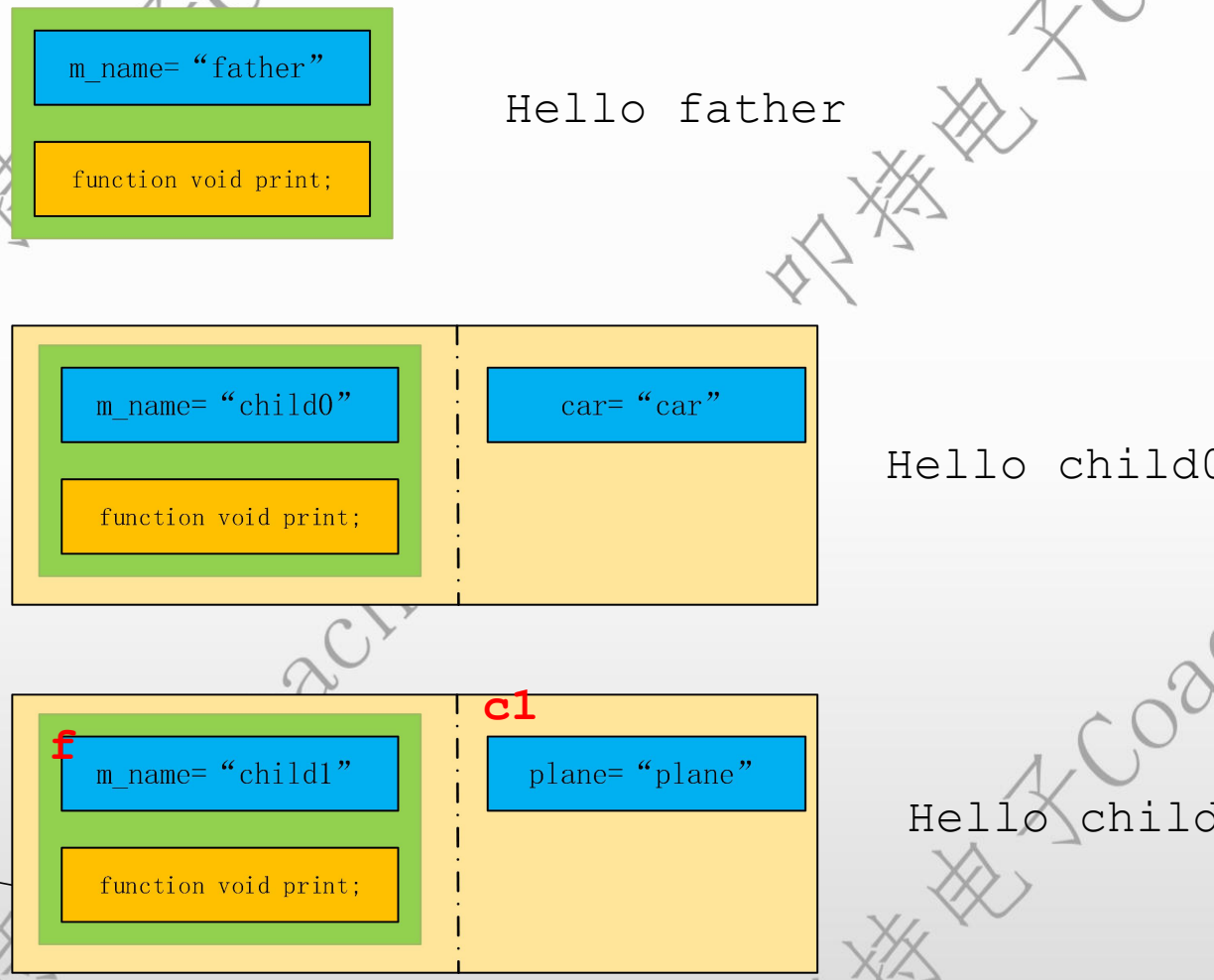
Hello child0

叩特电子Coachip

向上类型转换

```
module top;
  father f;
  child0 c0;
  child1 c1;

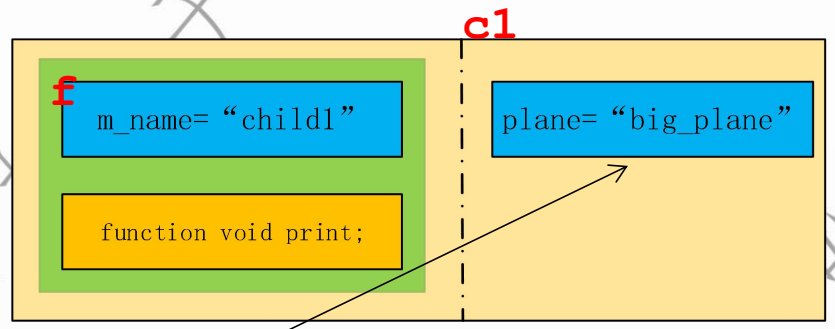
  initial begin
    f = new("father");
    f.print();
    c0 = new("child0");
    f = c0;
    f.print();
    c1 = new("child1");
    f = c1;
    f.print();
  end
endmodule : top
```



叩特电子Coachip

向下类型转换 cast

```
module top;
  father f;
  child0 c0;
  child1 c1;
  child1 c2;
  initial begin
    f = new("father");
    f.print();
    c0 = new("child0");
    f = c0;
    f.print();
    c1 = new("child1");
    f = c1;
    f.print();
    c1.plane="big_plane";
    $cast(c2,f);
    f.print();
    $write(", has %s",c2.plane)
  end
endmodule : top
```



f point to c1

c2 point to f

只有当前父类指针指向的对象和待转换对象的类型一致时，cast才会成功

叩特电子Coachip

练习----虚方法

```
class packet;
    int a =1;
    int b =2;
    function void display_a;
        $display("packet::a is %d",a);
    endfunction

    virtual function void display_b;
        $display("packet::b is %d",b);
    endfunction
endclass
```

```
class my_packet extends packet;
    int a =3;
    int b =4;
    function void display_a;
        $display("my_packet::a is %d",a);
    endfunction

    virtual function void display_b;
        $display("my_packet::b is %d",b);
    endfunction
endclass
```

```
module top;
    packet p1;
    my_packet p2;

    initial begin
        p1=new();
        p2=new();
        p1 = p2;

        p1.display_a;
        p1.display_b;
    end
endmodule
```

即特电子Coachip

练习----类型转换

```
class father;
  string m_name;

  function new (string name);
    m_name = name;
  endfunction

  function void print ();
    $display("Hello %s", m_name);
  endfunction
endclass : father
```

```
class child0 extends father;
  string car = "car";

  function new (string name);
    super.new(name);
  endfunction
endclass : child0
```

```
class child1 extends father;
  string plane = "plane";

  function new (string name);
    super.new(name);
  endfunction
endclass : child1
```

```
module top;
  father f;
  child0 c0;
  child1 c1;
  child1 c2;
  initial begin
    f = new("father");
    f.print();
    c0 = new("child0");
    f = c0;
    f.print();
    c1 = new("child1");
    f = c1;
    f.print();
    c1.plane="big_plane";
    $cast(c2,f);
    f.print();
    $write(", has %s",c2.plane);
  end
endmodule : top
```

即特电子Coachip

Q&A

昂特电子Coachip